

Econstellar Compute Engine: Verification Run Report, 2026-06-12

Avishek Bhandari

School of Humanities, Social Sciences and Management
Indian Institute of Technology Bhubaneswar
avishekb@iitbbs.ac.in

Run executed 2026-06-12T17:41:53.875Z (UTC)

Abstract

We report a complete verification run of the Econstellar compute engine, a continuously deployed econometric service for financial-market analysis in which every public method ships with a pre-registered, failable evaluation. The suite executed 26 method rows — spanning stationarity, unit roots and cointegration; volatility, long memory and spectra; information flow and transfer entropy; connectedness and networks; contagion channels and systemic risk; reproduction and formal verification — against the live service, each checked against an expected band fixed before the run and naming its full parameterisation and source: 26 passed, 0 failed, 0 pending. Every quantity traces to a live computation or a documented published result; the methods execute the original papers’ own software rather than reimplementations; and the document is generated mechanically from the run artefact, so that it may be regenerated and audited rather than taken on trust. Honest negatives, where present, are reported as the registered expectation: an empty network or a non-significant test is a result, not a blemish to manage.

Keywords: reproducible econometrics; transfer entropy; financial contagion; systemic risk; sandboxed computation; continuous verification.

JEL classification: C55, C58, C63, C88, G15.

1 The engine and the run

The engine is a sandboxed R/Node compute kernel on Google Cloud Run (service `shssm-compute`, region `asia-south1`), backed by an always-on workstation job server for long-running bootstrap and surrogate work. Each method executes the published package of its source paper rather than a reimplementation. The run below was produced by `evals/run-evals.mjs`, whose output `evals.json` is committed unedited beside the public pages and is the sole source of every number in this report.

Run timestamp	2026-06-12T17:41:53.875Z
Engine URL	https://shssm-compute-b7ui3oxaqq-el.a.run.app
Engine reachable at probe	no (probe) — see §9
Engine revision	not captured at this run’s probe
Registered methods	not captured at probe
Suite rows	26
Summary	26 pass / 0 fail / 0 pending
Engine repo commit	cd93e1f
Pages repo commit	ee163c0

2 Data and methods

The engine analyses daily log-returns of major markets. Two panels recur: `g20` (18 markets, 5036 daily observations, 306 directed pairs) and `g20_24` (24 series, adding commodities, 552 directed pairs). Information-flow estimators require stationary inputs, so returns — integrated of order zero — are used throughout, never price levels. The 26 method rows group into the families below; each names the published basis it runs.

Stationarity, unit roots and cointegration. Augmented Dickey–Fuller and KPSS tests, the Im–Pesaran–Shin and Levin–Lin–Chu panel unit-root tests, and Johansen’s trace test for cointegrating rank. The downstream information-flow estimators require inputs integrated of order zero, so log-returns are tested, never price levels.

Methods: `unit_root`, `live_unit_root`, `panel_unit_root`, `vecm`.

Volatility, long memory and spectra. GARCH(1,1) persistence, the Hurst exponent by de-trended fluctuation analysis, and a maximal-overlap discrete wavelet transform for scale-resolved variance and cross-market coherence.

Methods: `garch`, `dfa_hurst`, `namh_hurst`, `wavelet`, `wavelet_coherence`.

Information flow and transfer entropy. Transfer entropy [3] estimated by the Kraskov–Stögbauer–Grassberger nearest-neighbour scheme [1] in the Frenzel–Pompe conditional-mutual-information form [2], with significance assessed against IAAFT surrogates [4] and edges selected under Benjamini–Hochberg false-discovery control [5]; a wavelet–quantile variant resolves spillover by scale and quantile.

Methods: `ksg_te`, `ksg_robustness`, `namh_te`, `namh_pipeline`, `wqte`, `soch_profile`, `quantile_var`.

Connectedness and networks. Vector-autoregressive impulse responses, Granger-causal edges, and the Diebold–Yilmaz forecast-error-variance connectedness index [6], static and rolling, with a dynamic conditional-correlation companion and an `igraph` network surface.

Methods: `var_irf`, `granger`, `connectedness`, `spillover_rolling`, `network`, `rolling_dcc`.

Contagion channels and systemic risk. Attribution of crisis-window spillovers to economic channels (trade, finance and others; the global-financial-crisis decomposition is anchored to arXiv:2604.26546), and a daily systemic-risk index recomputed in full from the stored panel.

Methods: `channel_attribution`, `sri_daily`.

Reproduction and formal verification. A delta-verifier that re-runs the published estimators of the `namh` package [8] and reports the maximum absolute deviation from the archived output, and a seeded robustness badge [7] over the published SOCH symmetry test.

Methods: `namh_reproduce`, `soch_robustness`.

3 Results

Table 1 groups the 26 rows by family, placing each verified value beside the pre-registered band it was checked against: 26 of 26 fall within band (0 fail, 0 pending). The bands are not decorative. Several encode honest negatives: where the engine ships an empty false-discovery-controlled network or a bit-exact reproduction target, the null or the exact value is the correct outcome, and a populated network or a moved number would be the failure, not the pass.

Table 1: Verification suite grouped by method family: the verified value, the pre-registered band with its parameterisation, and pass/fail status. Rendered verbatim from `evals.json`.

Method	Value	Pre-registered band (parameterisation)	Status
Stationarity, unit roots and cointegration			
unit_root	-49.17960762	India ADF stat in [-55,-45] (verified -49.18)	pass
live_unit_root	—	live S&P: levels non-stationary, returns stationary (the stationarity gate)	pass
panel_unit_root	-77.25938898	panel stationarity rejected: IPS and LLC $\hat{\rho} \approx 0$ (this exact 5-market panel: IPS -77.26, LLC -51.79)	pass
vecm	3	Johansen rank 3 (verified trace 7265.97→1851.19)	pass
Volatility, long memory and spectra			
garch	0.99102203	India GARCH persistence in [0.95,1.0] (verified 0.991)	pass
dfa_hurst	0.54197024	India Hurst in [0.45,0.65] (verified 0.542)	pass
namh_hurst	0.48990576	g20.24 Gold H_mean in [0.4889,0.4909] (bit-exact verified 0.4899)	pass
wavelet	47.07	India d1 share of variance in [35,60]% (verified 47.07)	pass
wavelet_coherence	0.24851479666666665	mean coherence in [0.2,0.3] (verified 0.249, peak d6)	pass
Information flow and transfer entropy			
ksg_te	0.154234	USA→Japan full-sample KSG TE in [0.150,0.158] (verified 0.1542)	pass
ksg_robustness	0.7282	USA→Japan rank-1 in 8/8 (k,lag) configs; mean Spearman in [0.6,0.85] (verified 0.728)	pass
namh_te	-0.22322165	g20.24 window-1 te_mean in [-0.2242,-0.2222] (bit-exact verified -0.223222)	pass
namh_pipeline	-0.22322165	seeded end-to-end run (seed 42, L’Ecuyer-CMRG): FDR retains 0/552 under the paper’s own gate (degenerate network, honest amber); raw te_mean in [-0.2242,-0.2222]	pass
wqte	0.03910699	USA→India tau.05 aggregate spillover in [0.02,0.06] (verified 0.039)	pass
soch_profile	0.03910699	USA→India aggregate in [0.03,0.05] (verified 0.0391, peak d4)	pass
quantile_var	0.7006	USA is the top tail driver with positive net score (verified +0.70)	pass
Connectedness and networks			
var_irf	0.70532304	stable VAR: max companion root $\hat{\rho} < 1$, finite	pass
granger	6	6 significant directed edges (verified; USA out-degree 3)	pass
connectedness	30.25	total connectedness in [25,35]% (verified TCI 30.25)	pass
spillover_rolling	28.39	mean rolling TCI in [20,35]% (verified 28.4)	pass
network	0.5667	6-node graph, density in (0,1], communities ≥ 1	pass
rolling_dcc	0.2295	India–USA mean dynamic correlation in [0.15,0.31] (verified 0.23)	pass
Contagion channels and systemic risk			
channel_attribution	27.9	GFC: Trade 27.9% (Table 5, 0.000pp) and dominant == Trade	pass

Method	Value	Pre-registered band (parameterisation)	Status
<code>sri_daily</code>	0.00849	static-panel SRI = 0.00849 exactly $\pm 1e-5$ (first honest point, 306 pairs)	pass
Reproduction and formal verification			
<code>namh_reproduce</code>	1.55431223e-15	hurst GREEN: $\max \Delta \leq 1e-8$ over ≥ 400 finite cells, 0 finite/NA mismatches; te GREEN $\leq 1e-8$; fdr_network AMBER with 0/552 retained (D1: never green)	pass
<code>soch_robustness</code>	0.9911	SOCH-B badge, seeded tuple {seed 42 L’Ecuyer-CMRG, B=200, la8, advanced-8 = 28 pairs, grid $\tau \in \{0.05, 0.10\} \times J \in \{4, 5\} = 112$ points}: baseline $\tau=0.05/J=4$ holds 28/28 (paper ground truth reproduced) + pass_rate 0.9911 exact (111/112, matches Phase-31 0.991) - $\hat{\epsilon}$ badge robust	pass

4 Failures and pending rows

All 26 rows fall within their pre-registered bands. No band was widened and no extraction was altered after the run; the tables here are the unedited rendering of a single execution of the committed runner.

5 The deployment chain

This run closed a release cycle that moved the engine’s public registry from 23 to 26 methods. Three registrations landed together: `namh_pipeline` (a seeded end-to-end surrogate pipeline run as an asynchronous workstation job), `namh_reproduce` (a Δ -verifier that re-runs the published NAMH estimators and reports the maximum absolute deviation from the paper’s archived output), and `soch_robustness` (a badge-mode stability grid over the published SOCH-B symmetry test, the subject of Section 6).

The chain was: engine push (commits `37d4337..5d0dc80`, with the post-fix commit `f1616f2` following), source deploy to Cloud Run revision `shssm-compute-00033-wq2`, an environment-variable restore to revision `00034-zqz`, and — after the driver fix described below — a second deploy arriving at revision `shssm-compute-00036-11f`, which this report’s suite ran against. Two operational details are worth the record. First, the deploy script’s API-key lookup missed because the repository had moved; the documented remedy (an in-place service update, no rebuild) restored chat capability in under a minute, and the miss was loudly warned by the script itself — a designed failure surface behaving as designed. Second, the suite’s single-shot 15-second health probe once recorded a live engine as unreachable while all 26 method rows passed against it: the probe had raced a Cloud Run cold start. The probe now retries once with a longer window; the dead-engine path (a deliberately unreachable URL must produce a non-zero exit and honest red rows) still passes.

6 Case study: a stochastic gate caught by its own evaluation

The most instructive event of the run was a failure. The `soch_robustness` row re-runs the published SOCH-B direction-symmetry test (the `sochcontagion` package’s own `soch_test_symmetry`, GPL-3 — the paper’s code, not a reimplementaion) over a nuisance grid $\tau \in \{0.05, 0.10\} \times J \in \{4, 5\}$

on the eight advanced markets (28 unordered pairs per configuration, 112 grid points), with a 200-replication bootstrap null. Its expected band was pre-registered from the Phase-31 closure board: the baseline cell ($\tau = 0.05$, $J = 4$) holds 28/28 pairs — the paper’s ground truth — with an overall pass rate of at least 0.90.

The first genuine end-to-end execution of this driver returned 27/28 at baseline and an overall pass rate of 0.9821 (110/112), and the row went red. Investigation showed the driver carried no random-number-generator pin: the bootstrap null’s 95th-percentile threshold was stochastic across runs, and one near-boundary pair sat close enough to its threshold to flip between executions. The Phase-31 grid run had measured 0.9911 (111/112) — one grid point away — consistent with the same knife edge landing on the other side.

The repair followed the engine’s existing seeded-determinism precedent (`namh_pipeline`): the driver now pins `RNGkind("L'Ecuyer-CMRG")` with seed 42 [7], and reports both in its output. The band was not touched. A fresh anchor run (job `job_20260612_fe0c4f06`, 773.5s) returned baseline 28/28, pass rate 0.9911, badge *robust* — the paper’s ground truth and the Phase-31 measurement reproduced exactly under the pin. The full suite was then re-run from scratch, and its independently submitted job (job `job_20260612_c0398d0f`) returned the identical tuple, digit for digit: two independent executions now verify determinism end to end. The evaluation row was tightened, not widened — it now requires the exact seeded pass rate (0.9911), the exact baseline count (28/28), and the seed itself (42) to be echoed in the result.

Three points generalise beyond this engine. First, binary bootstrap-gated claims of the form “the pair holds symmetry” sit on resampling knife edges at modest replication counts; at $B = 200$, a single marginal pair is enough to move a headline count. A reproducibility statement for such a claim is meaningful only with the full tuple — seed, generator, replication count, filter, grid, and market set — stated. Second, grid resolution bounds the interpretability of badge rates: with 112 points, one flip moves the rate by $1/112 \approx 0.0089$, so 0.9821 versus 0.9911 is a one-point difference, not a qualitative change; counts should accompany rates. Third, the institutional lesson: the gate that failed was not noise to be routed around but the system working — a pre-registered band over a first-run driver surfaced a latent nondeterminism that would otherwise have produced intermittent, unexplainable reds in nightly operation.

7 Formal and epistemic layers

Two verification layers beyond the numerical suite are active. The formal layer (Lean 4 with `mathlib`, `toolchain v4.30.0`) machine-checks the SOCH spectral results: 11 of 12 declarations are Lean-accepted, including the unconditional closure of the product-Lorentzian spectral density proposition ($\|H(\omega)\|^2 = S(\omega)$) and an existence-and-uniqueness chain for the first-order-condition frequency; one declaration (the analytic half of the peak-location lemma) honestly carries `sorry` and is reported as unproved. The epistemic layer — a provenance-carrying claims ledger with twenty seeded claims spanning established, contested, and superseded states — is built and its public read surface is deployed; its data store provisioning remains a gated operator action, and until it runs the API degrades honestly, naming the missing dataset rather than fabricating an empty success.

8 Implications

For applied work, the run’s substantive anchors held without exception: the India ADF statistic (-49.18), the GARCH persistence ($\alpha + \beta = 0.991$), the named-panel IPS statistic (-77.26 on {India, USA, UK, China, Japan}), the NAMH transfer-entropy window mean (-0.22322165 , inside its

pre-registered band), the KSG USA→Japan transfer entropy (0.154), the KSG robustness Spearman (0.7282), the GFC trade-channel share (27.9%, arXiv:2604.26546), and the live systemic risk index (0.00849). The honest negatives also held: the FDR-gated NAMH network retains 0/552 edges and is reported as a degenerate network (amber), and the SOCH-C significance test remains not significant ($p = 0.105$) — both immutable facts of the record, not blemishes to be managed. The engine’s claim is not that its numbers are impressive but that they are traceable: every figure in this report resolves to a job identifier, a revision string, or a published artefact, and the pipeline that produced this document will produce the next one unattended.

Addendum (2026-06-13): GPU offload of the transfer-entropy search

The heaviest asynchronous methods in this run, `ksg_te` and `ksg_robustness`, spend their time in an $O(n^2)$ nearest-neighbour search under the Chebyshev norm — the Kraskov–Stoegbauer–Grassberger / Frenzel–Pompe transfer-entropy estimator. On the workstation this work had a latent failure mode: the drivers hard-coded `detectCores()-1` mclapply workers and ignored the job’s core request, so two concurrent robustness jobs forked forty-two workers over twenty-two logical cores with unpinned multi-threaded BLAS beneath them, pinned every core at 100%, and hung the machine mid-pipeline. That oversubscription was first removed at source — a single governed core budget, clamped last by a server-set hard ceiling, on every fork site — which makes the CPU path safe. The complementary step, recorded here, is to move the search off the CPU altogether.

The same primitive now runs on the workstation’s NVIDIA RTX 3000 Ada GPU through `numba.cuda`, a path that needed no new installation (the CUDA toolkit and `numba` were already present). The two methods dispatch to the GPU when a CUDA device is visible and fall back to the governed CPU path otherwise; a single environment switch forces the CPU path when wanted. The bar was the engine’s standing reproduce-integrity rule: the GPU must *equal* the CPU, not approximate it. The observed transfer entropy — the only quantity the evaluation suite gates — is held bit-exact. A dedicated gate measured a maximum absolute deviation of 7×10^{-16} between the GPU and the engine’s own CPU estimator (the six-decimal emitted figures identical) across embedding lags $\{1, 2\}$ and neighbour counts $\{4, 5\}$. The gate also surfaced a genuine floating-point subtlety worth recording: the engine’s one-dimensional neighbour count tests membership through the subtraction $v - \varepsilon$ rather than $|\Delta| < \varepsilon$, and so admits a boundary point that the textbook strict form rejects when $|\Delta|$ rounds to exactly ε . The GPU path was made to reproduce the engine’s behaviour, not the textbook’s, because the published figure — not the cleaner derivation — is the anchor a regression must protect.

Significance is unchanged in distribution. Neither transfer-entropy method seeds a generator, so the IAAFT surrogate p -value is a non-reproducible draw in both the CPU and GPU paths; the surrogate construction was vectorised into a batched Fourier transform on the host, and a pair’s observed estimate together with all ninety-nine of its surrogates now executes as one batched device launch, so that the per-launch and transfer overhead is paid once per pair rather than once per surrogate.

The test that matters is the workload that hung the machine, re-run on the new path: the full eighteen-market panel, all 306 directed pairs, ninety-nine IAAFT surrogates each — 30,600 neighbour searches. It completed in 35.6 minutes at 100% of a *single* core (peak resident set 388 MB), against a measured 289 s for a twelve-pair slice on the governed CPU path across roughly seven cores, which extrapolates to about two hours for the full panel: the GPU path is some $3.5\times$ faster while leaving the processor — and the other twenty-one cores — free for concurrent work. The substantive result is unmoved. The headline link, USA→Japan, returns transfer entropy 0.154234

to the digit; 106 of 306 directed links are significant at $p < 0.05$; and the strongest flows are all *into* Japan — from the United States (0.1542), Germany (0.1108), Canada (0.1075) and France (0.1030) — recovering the same information-sink structure the CPU path reports, now as a by-product of a run that no longer threatens the host.

Two points generalise. First, when a published estimator is ported to a faster device, the equivalence test must be written against the estimator’s own output, including its floating-point boundary conventions, rather than against the form in which it is usually written down; the device that is “more correct” is the wrong device if it moves a number the literature has already recorded. Second, massively parallel fan-out belongs on hardware built for it — a GPU’s thousands of lanes — rather than on an unbounded CPU fork that competes with every other job on the machine; the same arithmetic that saturated twenty-two cores now occupies one core and a few hundred megabytes of video memory, and runs faster for it.

9 Reproducibility and integrity

This report is generated mechanically from the run artefact and inherits its discipline. (i) Every row carries a pre-registered band naming its full parameterisation and a source, fixed before the run. (ii) `evals.json` is the output of one genuine run of the committed runner and is never hand-edited; to change a number one re-runs the suite, not the report. (iii) A band failure halts work — bands are never widened to pass, and only extraction plumbing (the harness misreading a result shape) may be calibrated, with the band untouched. (iv) Honest negatives are permanent: a non-significant test is reported as non-significant, an empty false-discovery network as empty [5], an unproved formal statement as unproved. (v) Asynchronous rows are real workstation jobs with persistent identifiers, or are reported pending — never fabricated.

Reproduction is graded rather than asserted. Deterministic estimators — the nearest-neighbour transfer entropy [1] and the Hurst panel of the published `namh` package [8] — are reproduced to a stated maximum absolute deviation and earn a green only on an exact match; stochastic surrogate stages, whose generators were unseeded in the original runs, are reported amber rather than dressed as exact. Where the engine seeds its own surrogate and bootstrap stages it uses a single declared generator [7], so a result is reproducible for a fixed seed and core count and the asymmetry with the unseeded published run is stated, not concealed. The engine and this verification apparatus are described in full in the accompanying system paper [9].

A Per-row provenance

Method	Source band / provenance	Tower job
<code>unit_root</code>	A2 + golden	sync
<code>var_irf</code>	A2 invariant (verified tuple: lag 7, root 0.705)	sync
<code>dfa_hurst</code>	A2 + golden	sync
<code>garch</code>	A2 + golden	sync
<code>wavelet</code>	A2 + golden	sync
<code>wqte</code>	A2	sync
<code>soch_profile</code>	A2	sync
<code>channel_attribution</code>	A2 + arXiv:2604.26546	sync
<code>namh_hurst</code>	A2 #45	sync
<code>namh_te</code>	A2 #45	sync
<code>vecm</code>	A2	sync
<code>granger</code>	A2	sync

Method	Source band / provenance	Tower job
panel_unit_root	A2 invariant — series set resolved 2026-06-11: the documented tuple is the {India,USA,UK,China,Japan} panel (live re-run -77.2594/-51.7933, exact)	sync
connectedness	A2	sync
network	A2 invariant	sync
rolling_dcc	A2	sync
wavelet_coherence	A2	sync
spillover_rolling	A2	sync
quantile_var	A2	sync
live_unit_root	A2 + invariant 4 (TE on I(0) returns, never levels)	sync
sri_daily	A2 Phase 30	sync
ksg_te	A2 + #39 anchor	job_20260612_92fa3f31
ksg_robustness	A2 G.3	job_20260612_8b3ef7a0
namh_reproduce	M1 Step 5 — measured 2026-06-12: same-machine 5.0e-16 (440 cells) / 1.7e-16; live-vs-cache $\approx 4.9\text{e-}9$ cross-BLAS (#45/#46)	sync
namh_pipeline	M1 Step 4 — determinism verified 2026-06-12 (same seed $\rightarrow$ identical surrogate p-values; different seed $\rightarrow$ different); FDR-empty = the published result (#46)	job_20260612_a697de03
soch_robustness	Phase 31 closure board + seeded anchor run job_20260612_fe0c4f06 (2026-06-12; an unseeded run flipped the baseline pair once, 27/28 — driver got the namh_pipeline seed pin, band never widened)	job_20260612_c0398d0f

References

- [1] Kraskov, A., Stögbauer, H., & Grassberger, P. (2004). Estimating mutual information. *Physical Review E*, 69(6), 066138.
- [2] Frenzel, S., & Pompe, B. (2007). Partial mutual information for coupling analysis of multivariate time series. *Physical Review Letters*, 99(20), 204101.
- [3] Schreiber, T. (2000). Measuring information transfer. *Physical Review Letters*, 85(2), 461–464.
- [4] Schreiber, T., & Schmitz, A. (1996). Improved surrogate data for nonlinearity tests. *Physical Review Letters*, 77(4), 635–638.
- [5] Benjamini, Y., & Hochberg, Y. (1995). Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B*, 57(1), 289–300.
- [6] Diebold, F. X., & Yilmaz, K. (2012). Better to give than to receive: predictive directional measurement of volatility spillovers. *International Journal of Forecasting*, 28(1), 57–66.
- [7] L’Ecuyer, P. (1999). Good parameters and implementations for combined multiple recursive random number generators. *Operations Research*, 47(1), 159–164.
- [8] Bhandari, A., & Sahu (2026). *namh*: Network Adaptive Market Hypothesis estimators. R package version 0.1.0 (GPL-3).
- [9] Bhandari, A. (2026). Econstellar: an open-source AI-augmented research engine for computational financial econometrics. *arXiv:2606.05705*.